

```
['CHANGED', 4, 8, 16]
>>> ham
[2, 4, 8, 16]
>>> id(bacon), id(ham)
(2356896337352, 2356896337480)
```

---

A visualization of the execution of this code is at <https://autbor.com/copycopy1>. The `ham` variable refers to a copied list object rather than the original list object referred to by `bacon`, so it doesn't suffer from this gotcha.

But just as variables are like labels or name tags rather than boxes that contain objects, lists also contain labels or name tags that refer to objects rather than the actual objects. If your list contains other lists, `copy.copy()` only copies the references to these inner lists. Enter the following into the interactive shell to see this problem:

```
>>> import copy
>>> bacon = [[1, 2], [3, 4]]
>>> ham = copy.copy(bacon)
>>> id(bacon), id(ham)
(2356896466248, 2356896375368)
>>> bacon.append('APPENDED')
>>> bacon
[[1, 2], [3, 4], 'APPENDED']
>>> ham
[[1, 2], [3, 4]]
>>> bacon[0][0] = 'CHANGED'
>>> bacon
[['CHANGED', 2], [3, 4], 'APPENDED']
>>> ham
[['CHANGED', 2], [3, 4]]
>>> id(bacon[0]), id(ham[0])
(2356896337480, 2356896337480)
```

---

A visualization of the execution of this code is at <https://autbor.com/copycopy2>. Although `bacon` and `ham` are two different list objects, they refer to the same `[1, 2]` and `[3, 4]` inner lists, so changes to these inner lists get reflected in both variables, even though we used `copy.copy()`. The solution is to use `copy.deepcopy()`, which will make copies of